Project 3: Word Embeddings, Comparative Analysis, Deep Learning Sentiment Classification, and Interactive Results UI

Kamal Ahmadov **Rufat Guliyev**

Spring 2026

Abstract

This report presents Project 3 for Azerbaijani NLP, covering six tasks: (1) corpus and matrix analysis, (2) Word2Vec training and intrinsic evaluation, (3) GloVe training and intrinsic evaluation, (4) side-by-side embedding comparison, (5) deep-learning sentiment experiments over a 3x5 grid (RNN/BiRNN/LSTM x Count/TF-IDF/PMI/Word2Vec/GloVe) with a baseline, and (6) report artifact generation. In addition, an interactive dashboard was built to explore outputs and embedding behavior through live word queries, analogy tests, and semantic maps.

1 Motivation

The project addresses two connected problems in Azerbaijani NLP. First, we need high-quality word representations that capture lexical similarity and semantic relations beyond sparse token counts. Second, we need to evaluate whether these representations improve downstream sentiment classification when used with recurrent neural architectures. The work therefore combines intrinsic embedding analysis (neighbors and analogies) with extrinsic task performance (3-class sentiment prediction).

2 Method and Deliverables

Project 3 was implemented as a dedicated pipeline under `project3_*` modules and executed through:

```
bash scripts/run_project3.sh
```

Main outputs are stored in:

- `outputs/project3/task1_matrices`
- `outputs/project3/task2_word2vec`
- `outputs/project3/task3_glove`
- `outputs/project3/task4_compare`
- `outputs/project3/task5_dl`
- `outputs/project3/task6_report`

Core entry points:

- `src/project3_task1_matrices.py`
- `src/project3_task2_word2vec.py`
- `src/project3_task3_glove.py`
- `src/project3_task4_compare.py`
- `src/project3_task5_dl.py`
- `src/project3_task6_report.py`
- `src/project3_dashboard.py` (interactive Streamlit UI)

3 Datasets and Experimental Setup

3.1 Corpus for Tasks 1–3

- Source file: `data/raw/corpus.csv`
- Provenance: same cleaned Azerbaijani corpus developed and used in Project 1 and Project 2.
- Text column: `text`
- Tokenization and sentence segmentation reuse existing project modules (`tokenize.py`, `sentence_segment.py`)

3.2 Sentiment Dataset for Tasks 4–5

- Source files: `data/external/train.csv`, `data/external/test.csv`
- Original dataset source: hajili/azerbaijani_review_sentiment_classification (Hugging Face).
- Note: local `train.csv`/`test.csv` are the project copies used by the pipeline.
- Text column: `content`
- Label column: `score`
- Label mode: 3-class sentiment (`negative`, `neutral`, `positive`)

3.3 Reproducibility Notes

- Full dependency set in `requirements.txt` (notably `torch`, `scipy`, `scikit-learn`, `flask`, `streamlit`, `plotly`).
- Imported `word2vec` and `GloVe` repositories are used for embedding training binaries.
- A smoke mode exists for fast integrity checks:

```
bash scripts/run_project3.sh --smoke
```

4 Task 1: Dataset and Matrix Analysis

4.1 Method

Task 1 builds corpus-level statistics and two matrix views:

- Term-document matrix (full sparse matrix + top-100 slice for visualization).
- Word-word co-occurrence matrix (top-100 terms for tractability).
- Frequency buckets:
  - frequent words: count ≥ 100
  - rare words: count ≤ 2

Artifacts include: `summary.json`, `word_frequency_distribution.csv`, `term_document_top_matrix.csv`, `word_word_top_matrix.csv`, and two heatmaps.

4.2 Results

Metric	Value
Documents	31,842
Sentences	813,319
Tokens	11,905,937
Vocabulary size	586,674
Frequent words (≥ 100)	12,436
Rare words (≤ 2)	398,599

Table 1: Task 1 dataset summary (`outputs/project3/task1_matrices/summary.json`).

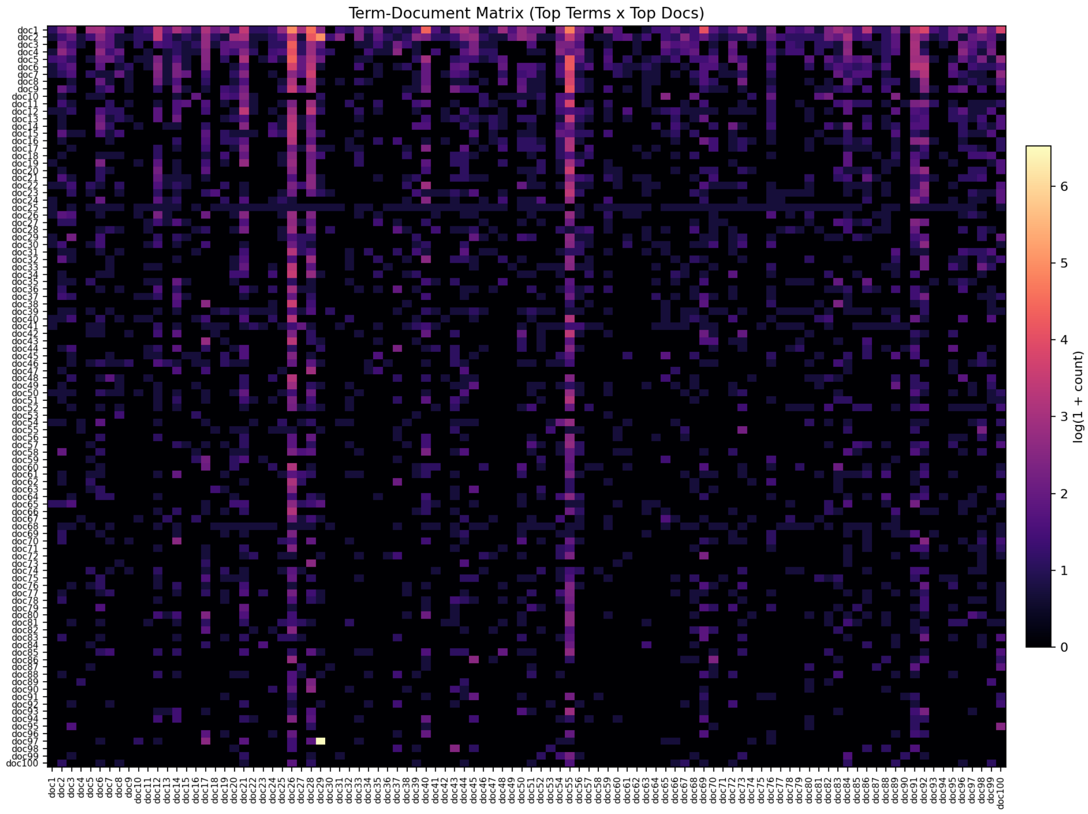

Figure 1: Task 1 term-document heatmap (log-scaled, top terms/docs).

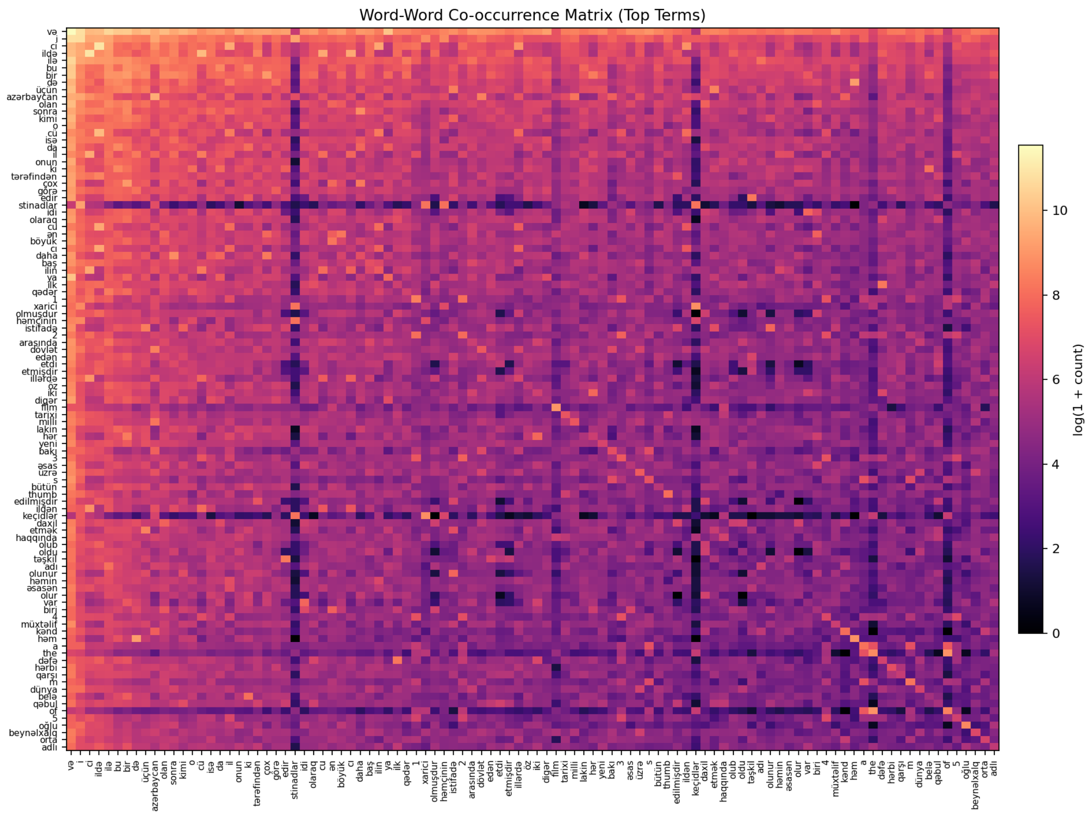

Figure 2: Task 1 word-word co-occurrence heatmap (log-scaled, top terms).

5 Task 2: Word2Vec Training and Analysis

5.1 Method

Training used the imported `word2vec/` C implementation with shared tokenized corpus input:

- Model: skip-gram
- Dimension: 200
- Window: 5
- Negative sampling: 10
- Min count: 5
- Iterations: 10

Intrinsic evaluation:

- Nearest-neighbor lookup for 10 target words.
- Fixed analogy list with automatic OOV skipping.

5.2 Results

Metric	Word2Vec
Vocabulary size	122,930
Vector dimension	200
Analogy questions (total)	5
Analogy questions evaluated	3
Analogy hit@1	0.6667
Analogy hit@k	0.6667

Table 2: Task 2 summary (`outputs/project3/task2_word2vec/summary.json`).

Representative artifacts:

- `outputs/project3/task2_word2vec/vectors.txt`
- `outputs/project3/task2_word2vec/nearest_neighbors.csv`
- `outputs/project3/task2_word2vec/analogy_results.csv`

5.3 Qualitative Synonym/Semantic Similarity Findings

The 10 target words were: `azərbaycan, şəhər, tarix, dövlət, dünya, insan, dil, elm, mədəniyyət, film`.

Observed nearest-neighbor behavior:

- Strong morphological/lemma consistency: `şəhər` → `şəhəri`, `tarix` → `tarixi`, `insan` → `insanın`, `dil` → `dili`.
- Reasonable semantic relation in some cases: `dünya` → `avropa`.
- Noticeable noise in others: `elm` → `2016,400` and `azərbaycan` → `uzunömürlülər`.

Overall, Word2Vec showed good local structure for common lexical/morphological variants but still contained noisy neighbors for some targets.

5.4 Vector Arithmetic Analysis (Word2Vec)

Analogy evaluation shows both meaningful regularities and failure cases:

- Correct: `kişi:qadın::oğlan:qız` (hit).
- Correct: `bakı:azərbaycan::ankara:türkiyə` (hit).
- Incorrect semantic relation: `ata:ana::oğul:qız` predicted `istərəm`.
- OOV-limited cases were skipped (e.g., `ən_böyük`, `daha_pis`).

This indicates that geographical and gender analogies are partially captured, while more nuanced family/degree relations remain unstable.

6 Task 3: GloVe Training and Analysis

6.1 Method

Training used the imported `GloVe/` C implementation:

- Dimension: 200
- Window size: 10
- Min count: 5
- Max iterations: 20
- $x_{\max} = 10$, $\eta = 0.05$, $\alpha = 0.75$

Evaluation follows the same nearest-neighbor and analogy protocol as Task 2.

6.2 Results

Metric	**GloVe**
Vocabulary size	122,930
Vector dimension	200
Analogy questions (total)	5
Analogy questions evaluated	4
Analogy hit@1	0.7500
Analogy hit@k	0.7500

Table 3: Task 3 summary (`outputs/project3/task3_glove/summary.json`).

Representative artifacts:

- `outputs/project3/task3_glove/vectors.txt`
- `outputs/project3/task3_glove/vocab.txt`
- `outputs/project3/task3_glove/nearest_neighbors.csv`
- `outputs/project3/task3_glove/analogy_results.csv`

6.3 Qualitative Synonym/Semantic Similarity Findings

For the same 10 target words, GloVe also learned strong morphological neighbors:

- `şəhər` → `şəhəri`, `tarix` → `tarixi`, `insan` → `insanın`, `dil` → `dili`.
- Domain-like relation: `elm` → `mədəniyyət`.
- Noise/out-of-domain neighbors also appeared for some words (e.g., `azərbaycan` → `mars-print`).

Compared to Word2Vec, GloVe produced stronger analogy scores in this run but still exhibited noisy top neighbors for some targets.

6.4 Vector Arithmetic Analysis (GloVe)

Analogy behavior:

- Correct: `kişi:qadın::oğlan:qız` (hit).
- Correct: `bakı:azərbaycan::ankara:türkiyə` (hit).
- Correct directional relation: `şimal:cənub::şərq:qərb` (hit).
- Incorrect: `ata:ana::oğul:qız` predicted `istərəm`.

Patterns suggest that broad geographic/directional regularities are represented better than finer family-role analogies.

7 Task 4: Word2Vec vs GloVe Comparison

7.1 Method

Task 4 compares Task 2 and Task 3 with consistent intrinsic metrics:

- Analogy hit@1 and hit@k (from Task 2/3 summaries)
- Average top-3 neighbor cosine
- Coverage ratio on sentiment vocabulary from local `data/external/train.csv`+`test.csv` (project copy of the Hugging Face sentiment dataset above)

7.2 Results

Model	Hit@1	Hit@k	Avg top-3 cosine	Coverage ratio
Word2Vec	0.6667	0.6667	0.6619	0.0749
GloVe	0.7500	0.7500	0.6217	0.0749

Table 4: Task 4 embedding comparison (`outputs/project3/task4_compare/comparison.csv`).

Interpretation:

- GloVe wins analogy hit@k.
- Word2Vec wins average neighbor cosine.
- Coverage is identical for both models on this sentiment vocabulary.
- Final choice depends on objective (analogy quality vs. local neighborhood geometry).

8 Task 5: Deep Learning Sentiment Experiments (3x5 Grid)

8.1 Method

Task 5 runs 15 neural experiments plus one baseline:

- Architectures: RNN, BiRNN, LSTM
- Feature families: `count_svd`, `tfidf_svd`, `pmi_svd`, `word2vec`, `glove`
- Baseline: Logistic Regression + TF-IDF
- Primary metric: macro-F1
- Early stopping with fixed seed (42)

Dataset usage:

- Train total: 127,537
- Test total: 31,885
- Effective split in run: 114,783 train / 12,754 val / 31,885 test
- Label order: negative, neutral, positive

8.2 Results

Rank	Experiment	Accuracy	Macro-F1	Weighted-F1
1	`lstm__pmi_svd`	0.8225	0.5684	0.8594
2	`rnn__pmi_svd`	0.8396	0.5678	0.8641
3	`birnn__pmi_svd`	0.8170	0.5600	0.8541
4	`logreg_baseline__tfidf`	0.8271	0.5588	0.8582
5	`lstm__tfidf_svd`	0.8156	0.5488	0.8489

Table 5: Top Task 5 experiments by macro-F1 (`outputs/project3/task5_dl/leaderboard.csv`).

Best overall configuration:

- Experiment: `lstm__pmi_svd`
- Accuracy: 0.8225
- Macro-F1: 0.5684
- Weighted-F1: 0.8594

Baseline (LogReg + TF-IDF):

- Accuracy: 0.8271
- Macro-F1: 0.5588
- Weighted-F1: 0.8582

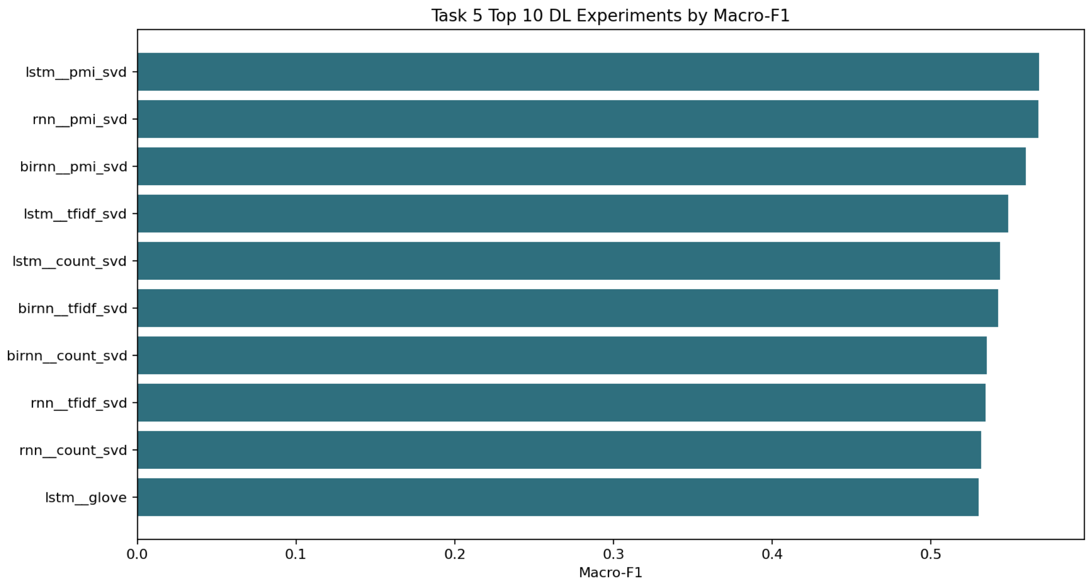

Figure 3: Task 5 top experiments by macro-F1 (from presentation bundle).

Additional Task 5 artifacts:

- `outputs/project3/task5_dl/metrics.csv`
- `outputs/project3/task5_dl/confusion_matrices.json`
- `outputs/project3/task5_dl/classification_reports.txt`

9 Task 6: Consolidated Report Artifacts

Task 6 aggregates key outputs into report-ready tables and summary files:

- `outputs/project3/task6_report/embedding_comparison_table.csv`
- `outputs/project3/task6_report/dl_results_table.csv`
- `outputs/project3/task6_report/report_artifacts.md`
- `outputs/project3/task6_report/summary.json`

Additionally, a presentation package is available at:

- `outputs/project3/presentation_bundle/`
- `outputs/project3/presentation_bundle.zip`

10 Interactive UI and Exploration

Two interfaces are available:

- **Primary:** Streamlit dashboard (`src/project3_dashboard.py`)
- **Legacy:** Flask+HTML dashboard (`src/project3_results_ui.py`, `src/project3_results_ui.html`)

Recommended launch command:

```
streamlit run src/project3_dashboard.py -- --output-root outputs/project3
```

The Streamlit UI includes a `Playground` where users can:

- query nearest neighbors for custom words,
- run custom analogies ($a : b :: c :?$),
- inspect cosine similarity between typed words,
- blend vectors interactively and inspect nearest results,
- generate 2D semantic maps for custom word sets.

11 Experiments and Error Analysis

11.1 Main Experimental Outcomes

- Intrinsic embedding comparison favored GloVe on analogy hit@k, while Word2Vec led on average top-3 neighbor cosine.
- In downstream sentiment experiments, the best system was `lstm__pmi_svd` with macro-F1 0.5684.
- The required baseline (`logreg_baseline__tfidf`) was included and is close to the best neural run in macro-F1.

11.2 Error Patterns

- OOV-driven skips in analogy sets reduce effective sample size for intrinsic evaluation.
- Some nearest-neighbor lists include noisy tokens or cross-script artifacts, especially for high-frequency entities.
- The sentiment dataset is strongly imbalanced toward the positive class, which limits neutral-class performance and lowers macro-F1.

12 Discussion

12.1 What Worked Well

- End-to-end reproducible pipeline with clearly separated task outputs.
- Both Word2Vec and GloVe trained successfully on the same tokenized corpus.
- Comparable intrinsic evaluation protocol enabled fair model comparison.
- Full 3x5 deep-learning grid executed and summarized with baseline.
- Outputs were transformed into report and presentation bundles.

12.2 Observed Constraints

- The sentiment dataset is highly imbalanced toward the positive class, which suppresses neutral-class performance.
- Intrinsic analogy set is intentionally small and includes OOV-sensitive items.
- Embedding coverage of Task 5 model vocabulary for pre-trained vectors is limited in this run (reported in Task 5 summary metadata).

13 Conclusion

Project 3 was completed with all requested tasks and artifacts: matrix analysis, Word2Vec/GloVe training and evaluation, comparative intrinsic analysis, full deep-learning grid, consolidated report outputs, and an interactive UI for result exploration. Final results show a clear tradeoff:

GloVe leads on analogy hit@k while Word2Vec leads on neighborhood cosine quality; on downstream sentiment classification, the best run is `lstm__pmi_svd` with macro-F1 0.5684.